
CAIM Sessió 5: MapReduce i agrupació de documents

Ferriol Falip Torras i Martí Puerta Juvés

12/12/2024

1 Introducció

En aquesta pràctica simulem el funcionament de Hadoop Map-Reduce utilitzant la llibreria MRJob de Python per implementar l'algorisme K-Means Clustering aplicat a documents.

A l'algorisme de K-Means, les dades es projecten com a punts en un espai multidimensional, i aquests s'assignen al clúster més proper en funció de la distància al centroide del clúster. En aquesta pràctica, representem cada document com un vector booleà basat en tokens, mentre que els centroides es construeixen utilitzant la freqüència de tokens de tots els documents assignats al centroide. Per calcular la similitud entre document i centroide utilitzem la similitud de Jaccard.

Relacionat amb la implementació, no se'ns van presentar grans dificultats, ja que el procés consistia principalment a comprendre el funcionament tant de MapReduce com de l'algorisme K-Means. No obstant això, el més complex va ser calcular la similitud de Jaccard entre un vector booleà i un clúster, ja que no eren dades del mateix tipus, fet que va requerir un tractament addicional per garantir la compatibilitat dels càlculs.

2 Experimentació

2.1 Freqüència mínima i màxima

Primer de tot anem a veure com afecten els paràmetres freqüència mínima i màxima a l'extracció de les dades. A continuació es mostra la quantitat de paraules extretes segons el rang.

Freqüència mínima i màxima	Mida vocabulari
0.01 - 0.1	1602
0.1 - 0.3	258
0.3 - 0.7	35
0.7 - 0.9	3

Taula 1: Quantitat de paraules al fitxer vocabulary.txt segons la freqüència mínima i màxima.

Com és d'esperar, si agafem freqüències molt baixes com, per exemple, entre 0.01 i 0.1, obtenim moltes paraules, però ens pot passar que aquestes no siguin prou identificatives per poder agrupar bé els documents. Per altra banda, si agafem freqüències molt altes, obtenim molt poques paraules, ja que només agafem aquelles paraules que apareixen de forma molt elevada a tots els documents, la qual cosa donarà com a resultat que tots els documents s'agruparan com iguals. Anem a veure si això és realment així mirant el nombre de clústers en què s'agrupen segons aquestes freqüències. Començarem amb 10 clústers i mirarem si en 5 iteracions arriben a convergir i quants clústers finals obtenim. També observarem el temps per iteració, cal destacar que les execucions s'estan fent des d'una màquina virtual amb capacitats limitades, la qual cosa pot incrementar els temps d'execució. Podem veure com per 0.7 - 0.9 el programa ha convergit i ha reduït de 10 a 4 clústers.

Freqüència mínima i màxima		It0	It1	It2	It3	It4	It5
0.01-0.1	clústers	10	10	10	10	10	10
	temps	-	16.49	438.06	317.3	308.49	310.69
0.1 - 0.3	clústers	10	10	10	10	10	10
	temps	-	14.7	57.86	54.85	54.56	54.12
0.3 - 0.7	clústers	10	10	10	10	10	10
	temps	-	7.72	9.6	10.14	10.21	10.06
0.7-0.9	clústers	10	4	4	-	-	-
	temps	-	2.99	2.54	-	-	-

Taula 2: Evolució del nombre de clústers i temps segons la freqüència mínima i màxima de les paraules del vocabulari.

Pel que fa a la resta de freqüències no observem cap reducció dels clústers, però donat que totes tenen vocabularis de més de 10 paraules és possible organitzar-les en més clústers, amb més iteracions potser es podria observar alguna reducció, ja que els programes no han convergit. Pel que fa al temps veiem com la mida del vocabulari influeix en el temps i que la primera iteració tarda significativament menys. Aquest últim fet es pot deure al fet que la primera iteració es treballa amb uns prototips més simples generats de forma aleatòria.

Finalment, hem decidit fixar les freqüències en 0.1 i 0.3, ja que considerem que és prou baix perquè el vocabulari sigui prou dispers, però prou alt perquè no ho sigui massa.

2.2 Temps segons el nombre de cores

Anem a veure ara com el nombre de cores pot influir en el temps. Treballarem amb dos conjunts de dades, un que serà de mida 100 i un altre de mida 250. Utilitzarem la mitjana de temps de les iteracions excloent la primera, pel que hem vist anteriorment.

Mida	Cores			
	1	2	4	8
100	21.4	22.83	23.96	27.23
250	49.07	51.18	51.98	52.76

Taula 3: Temps d’execució mitjana de cada iteració amb diferent nombre de cores.

Podem observar com no sembla que el nombre de cores afecti significativament al temps, sinó que més aviat l’empitjora. No estem segurs de si això és perquè realment és així, per algun error nostre o per culpa de la màquina virtual.

2.3 Execució final

A continuació veurem què passa quan augmentem considerablement el nombre d’iteracions. Treballarem amb el conjunt de mida 100, ja que va significativament més ràpid. Començarem amb 20 prototips i farem 30 iteracions (sempre que no convergeixi abans).

Si analitzem algun dels clústers obtinguts podem veure que parlen de temes més o menys concrets.

- CLASS2: ['comput', 'effici', 'approach', 'develop', 'demonstr', 'requir', 'applic', 'simul', 'process', 'compar', 'work', 'new', 'algorithm', 'problem']
- CLASS9: ['distribut', 'simul', 'scale', 'measur', 'function', 'compar', 'paramet', 'analysi', 'depend', 'includ', 'relat', 'rang', 'signific', 'over']
- CLASS19 ['limit', 'deriv', 'order', 'obtain', 'case', 'non', 'form', 'onli', 'space', 'consid', 'paramet', 'while', 'term', 'point']

Per exemple la classe 2 parla de temes de computació, la classe 9 sembla que parla de temes de probabilitat i anàlisis i la classe 9 de matemàtiques.

3 Conclusions

En resum, segons el valor del rang de freqüències que agafem obtindrem resultats molt diferents, en el nostre cas, utilitzant l’interval 0.1-0.3 el per a un bon equilibri entre dispersió i agrupació. El mètode MapReduce amb K-Means permet identificar temàtiques concretes com computació, probabilitat i matemàtiques, tot i que iteracions addicionals milloren la precisió amb un cost notable en temps d’execució. A més, l’ús de múltiples cores no sembla que redueix-hi significativament els temps, probablement per limitacions de la màquina virtual utilitzada.